

Práctica Temas 2 y 3

Algoritmos y Estructuras de Datos I

Antonio López García

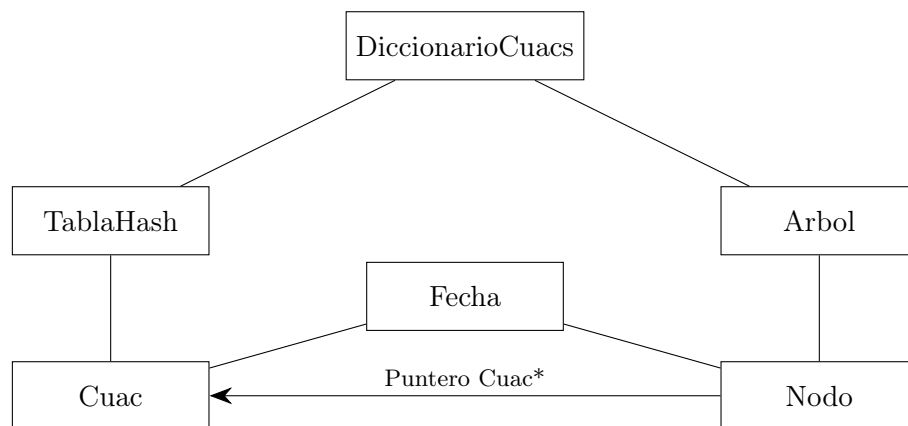
Curso: 2025/26

Índice

1	Clases definidas y dependencias	3
2	Módulos implementados, archivos de cada módulo	3
3	Tabla de dispersión utilizada. Explicación y justificación de las decisiones tomadas	4
4	Árbol utilizado. Explicación y justificación de las decisiones tomadas	6
5	Variables globales utilizadas (en su caso)	9
6	Otras decisiones de diseño relevantes	9
7	Resumen de tiempo dedicado a cada apartado	15
8	Conclusiones	16

1 Clases definidas y dependencias

(no incluir el código solo nombres de clases y dependencias)



Como se observa en el diagrama, la clase principal del proyecto es la de **DiccionarioCuacs**, la cual actúa como fachada, ya que es la que se encarga de implementar las clases más complejas como **TablaHash** o **Arbol** ofreciendo una interfaz más sencilla y haciendo el main mucho más sencillo. Este diccionario es como un coordinador que distribuye los mensajes recibidos, el cual cuando recibe un mensaje, es el encargado de enviarlo a la tabla, y luego referenciarlo al árbol, es decir, que delega el almacenamiento a la tabla y la indexación al árbol.

Luego tenemos **TablaHash**, una tabla de dispersión abierta, la cual utiliza un array dinámico de listas enlazadas `list<Cuac>` y es la encargada de almacenar en memoria los **Cuac**, y las colisiones de nombres de usuarios, que es lo que puede coincidir, se resuelve con el array de listas como he dicho antes.

Aparte tenemos también el **Arbol**, el cual implementa un AVL en mi caso. Sirve para los comandos `date` y `last` con complejidad logarítmica, porque si no tendríamos que recorrer toda la tabla y lista de cuacs para poder acceder a la fecha y poder resolver estos comandos. Lo que se hace con el árbol es cuando se guarda un **Cuac** en la tabla, ésta devuelve un puntero **Cuac***, el cual es tomado por **DiccionarioCuacs** y se lo pasa al árbol para poder guardar ese puntero referenciando a la fecha de ese **Cuac** y hacer estos accesos mucho más rápido.

Muy ligado al árbol tenemos la clase **Nodo**, la cual representa como bien dice su nombre, cada nodo del árbol AVL. Como puede darse que existan varios mensajes en el mismo instante de tiempo, para poder acceder a cada uno de ellos de manera independiente y que no haya problemas al asignarle a cada uno su nodo dentro del árbol, lo que hacemos es que cada nodo tenga una lista `list<Cuac*>` con un puntero a la fecha de cada **Cuac** guardado. Como bien he dicho, el nodo guarda punteros a los mensajes guardados en memoria en la **TablaHash**. Es por esto por lo que vemos la flecha que va desde **Nodo** a **Cuac**, porque se guardan punteros a estos mismos.

Con la clase **Cuac**, lo que hacemos es que a partir de un mensaje, modelamos su información como nombre de usuario, texto y fecha.

Por último la clase **fecha** nos sirve para comparar fechas de **Cuacs** como con `es_anterior()` y también nos sirve para facilitar la lógica del árbol.

2 Módulos implementados, archivos de cada módulo

(no incluir el código, solo nombres de módulos y sus dependencias)

Debido a la complejidad del proyecto, a partir del ejercicio 005, se ha ido estructurando en diferentes módulos, separando así definiciones de clases y sus implementaciones, así como del programa principal el cual ha recibido el nombre del ejercicio seguido de “.cpp” para diferenciarlo de ejercicios anteriores debido a los pequeños cambios que podía haber con el intérprete de comandos de un ejercicio a otro.

El módulo principal, es decir la interfaz, que gestiona la interacción con el usuario:

Archivo: “NºdeEjercicio”.cpp

Dependencias: DiccionarioCuacs.h, para controlar el sistema. Fecha.h y Cuac.h para las variables temporales de la entrada.

Luego tenemos la fachada explicada antes, el controlador:

Archivos: DiccionarioCuacs.h, DiccionarioCuacs.cpp

Dependencias: TablasHash.h, Arbol.h, Cuac.h

Luego tenemos los módulos que manejan y almacenan la información:

Almacenamiento físico (tabla de dispersión abierta):

Archivos: TablaHash.h, TablaHash.cpp

Dependencias: Cuac.h

También tenemos el que implementa el índice cronológico y equilibrado (Árbol AVL)

Archivos: Arbol.h, Arbol.cpp

Dependencias: Nodo.h

Y luego la unidad del árbol (Nodo)

Archivos: Nodo.h, Nodo.cpp

Dependencias: Cuac.h, Fecha.h

Por último tenemos los módulos de los datos:

El de los Cuac:

Archivos: Cuac.h, Cuac.cpp

Dependencias: Fecha.h

Y para terminar el del tiempo (Fecha):

Archivo: Fecha.h, Fecha.cpp

Éste es un módulo independiente por lo que no tiene dependencias.

Para la compilación tenemos el Makefile, que pasa desapercibido pero sin él no habría programa. El makefile es un fichero en el que le dice al compilador, cómo debe ser compilado el programa y en qué orden, con los diferentes módulos para que trabajen de manera simultánea y generando así los archivos objeto que son los .o y enlazarlos en el ejecutable a.out.

3 Tabla de dispersión utilizada. Explicación y justificación de las decisiones tomadas

El principal problema que experimentaba nuestro proyecto era que, a la hora de recibir muchos Cuacs, como se esperaba, colapsaba, por lo que fue necesario implementar nuevas estructuras de almacenamiento y manejo de datos como fueron las tablas de dispersión en este caso para almacenar los usuarios. Así también conseguíamos reducir la complejidad del comando follow (que buscaba mensajes por usuario), de $O(N)$ en el que teníamos que leer todos los Cuacs,

pasamos a $O(1)$, con un acceso directo gracias a esta tabla de dispersión, en la que almacenamos los Cuacs por usuario.

Como bien sabemos, a la hora de definir una tabla tenemos la opción de que sea de dispersión abierta o de que sea de dispersión cerrada. En nuestro caso escogimos la dispersión abierta con un array de listas enlazadas por cubeta, definido como `list<Cuac>*T`, esto lo que hace es que `T` apunte inicialmente a una lista vacía, y posteriormente con el constructor de la tabla `T = new list<Cuac>[M]`; creamos una lista y nuestro puntero `T` apunta al primer valor de esta lista. Se usa como puntero para poder pedir memoria dinámica que funciona mejor para estructuras grandes, y evitamos problemas como el desbordamiento de pila cuando la memoria que hay cargada en pila, es más pequeña que el tamaño de la lista.

Nuestra principal motivación para elegir dispersión abierta fue que no conocíamos el número final de elementos, ya que, en la dispersión cerrada, cuando la tabla se llena, el rendimiento cae muy notoriamente y requiere de un redimensionamiento y funciones de redispersión (para las colisiones) con un coste elevado. Por otro lado, con la dispersión abierta, el factor de carga (lo que se llena la tabla N/M), puede superar el 1.0 sin que colapse, ya que las colisiones simplemente se añaden al final de la lista de la cubeta correspondiente, que, aunque degrade el rendimiento, lo hace de manera mucho más suave que en la de dispersión cerrada.

A la hora de guardar los datos en las listas de las cubetas, también se nos plantearon dos opciones, las cuales eran: la primera almacenar directamente los objetos `Cuac` en la lista, o, almacenar un par de `Usuario` y el mensaje `Cuac`. Nosotros elegimos la primera, ya que, aunque la opción de almacenar un par puede agrupar mejor los mensajes de un usuario, a la hora de hacer la inserción, tendríamos que ver si existe en la lista de la cubeta, y luego insertar en su sub-lista, lo que requiere de más accesos. Sin embargo, la primera opción permite una inserción inmediata $O(1)$ en la lista de la cubeta, lo que simplifica el código y reduce el uso de memoria.

Para la función de dispersión se ha utilizado una función de suma posicional, ya que como se nos recomendaba, producía un reparto lo más uniforme posible y se podía calcular rápidamente, evitando funciones como la suma de códigos ASCII o su multiplicación. La función es:

$$H(s) = (s[0] \cdot B^{n-1} + s[1] \cdot B^{n-2} + \dots + s[n-1]) \bmod M$$

Nosotros teníamos clara la idea de la función sobre el papel, pero sinceramente, no teníamos claro cómo implementarla en el código por lo que recurrimos a la IA. Nos recomendó que pusiéramos: `n = (n*B + c) % M` dentro de un `for(unsigned char c:nombre)`, en el que la `c` representa cada carácter del nombre. Lo que hace es con cada carácter sumarle la multiplicación de `B` que es un número primo, en nuestro caso $B = 131$, con la multiplicación del carácter anterior para así, dependiendo del orden de las letras en la palabra, darnos un resultado u otro. El mod `M` como sabemos, es para que nos de una cubeta dentro del rango de la tabla, en el que `M` es el tamaño de la tabla. Además, si nos fijamos bien, ponemos `unsigned char` en vez de `char` solamente, esto es porque se nos advirtió que ciertos caracteres como tildes podrían producir valores negativos y con ello accesos a índices negativos del array y resultados erróneos, por eso los tratamos sin signo (`unsigned`).

Por lo que, en definitiva, la función de dispersión asigna una cubeta a cada usuario basándose en su nombre. Idealmente, cada usuario tendría su propia cubeta, pero debido a que el número de posibles nombres es infinito y la tabla es finita, ocurren colisiones. En ese caso, dos usuarios distintos pueden compartir la misma cubeta. Por eso se usa una lista enlazada dentro de cada cubeta, para poder almacenar los mensajes de todos los usuarios que hayan caído en esa misma posición.

Además elegimos un tamaño de tabla $M = 200003$, ya que es un número primo lo suficientemente grande para minimizar las colisiones.

Como ya hemos visto, aunque una tabla dinámica se adapta mejor, es mas costosa de mantener ya que al completarse la tabla, implica crear una nueva tabla y volver a calcular la posición de todos los elementos, como ya hemos mencionado antes, entonces optamos por reservar una tabla lo suficientemente grande para evitar el tener que redimensionar y evitar así el aumento en el tiempo de ejecución.

Aunque pueda resultar contradictorio, no lo es. Lo que hemos hecho es reservar un tamaño predeterminado para la tabla, M , aunque la reserva de memoria se hace de forma dinámica, con `new list<Cuac>[M]`.

También tenemos el tamaño de las cubetas en la que utilizamos también memoria dinámica con `list`, que a diferencia del array principal (la tabla) tienen un tamaño variable, creciendo solamente cuando es necesario, por esto es por lo que hemos cogido dispersión abierta, ya que no nos obliga a la redimensión, como hemos mencionado anteriormente.

Reiteramos este punto para que se vea claramente la diferencia del uso de memoria entre la tabla y las listas de las cubetas, así como la manera en la que se reserva la memoria en la tabla.

4 Árbol utilizado. Explicación y justificación de las decisiones tomadas

Una vez implementada la tabla, vemos que nos resulta muy útil para las operaciones `follow`, pero como carece de orden intrínseco, hace ineficientes las búsquedas cronológicas (por fecha). Por lo cual, los comandos `last` (los últimos N mensajes) y `date` (mensajes en un rango de fechas), requerirían recorrer toda la tabla, para mirar todos los mensajes y todos sus datos y poder ofrecer una respuesta, lo cual tendría una complejidad $O(N)$. Es por esto que, para optimizar la consulta por fecha, usamos los árboles.

Nosotros hemos escogido los Árboles Binarios de Búsqueda Balanceados (AVL), ya que los árboles Trie, como van por prefijos para acceder a una sola fecha, tendríamos que hacer muchos accesos y cada hoja sería muy profunda, ya que cada número de la fecha sería un prefijo. Además, para los comandos `a` tratar, tendríamos que ir por ramas disjuntas para los valores intermedios, lo cual complica mucho el ejercicio, y pierde eficiencia.

Por otro lado, tenemos los árboles B, con los que realmente a la hora de valorarlos, nos perdíamos en su implementación, ya que, al tener varias claves por nodo y árboles poco profundos y muy anchos, suponían una complicación de implementación. A su vez, descartamos los Árboles Binarios de Búsqueda (ABB) ya que como es obvio, a la hora de ir insertando Cuacs, en orden cronológico, degeneraría rápidamente en una lista enlazada, con una complejidad de inserción y búsqueda de $O(N)$ con lo cual no se resolvería el problema a tratar.

Sin embargo, los AVL aparte de ser los que a nosotros nos parecen los más sencillos e intuitivos, nos garantiza mediante sus rotaciones el balanceo del árbol y que mantenga siempre su orden de $\log N$. Por lo que tanto la inserción como las consultas para `date` y `last` se ejecuten de manera eficiente.

Otro impedimento que encontramos fue, a la hora de gestionar múltiples mensajes que ocurren en un instante de tiempo exacto, ya que como podemos deducir, irían al mismo nodo, tendríamos una colisión temporal. ¿Entonces que hacemos al respecto?

Para ello, se nos plantearon dos posibles soluciones:

Un nodo por mensaje, en el que cada nodo tenía un puntero a un Cuac, lo cual podría parecer lo más lógico y sencillo, pero ante fechas iguales, el nuevo nodo se insertaría en el hijo izquierdo o derecho. Esto aumentaría la altura del árbol innecesariamente, ya que al tener un nodo por

mensaje, el número de nodos aumenta notoriamente y al depender la complejidad de búsqueda de la altura ($\log N$), ralentizaría las operaciones de búsqueda y rotación.

La otra opción, por la que nos decantamos nosotros, era que cada nodo del árbol representara un instante de tiempo único, actuando como un contenedor para todos los mensajes ocurridos en ese instante específico, concretamente, una lista (`list<Cuac*>`) de punteros a los Cuacs.

En `Nodo.h`, donde definimos la clase `nodo`, podemos ver su declaración, `std::list<Cuac*>`. Agrupando los mensajes coincidentes en una sola estructura (una lista en este caso), reducimos el número total de nodos que debe gestionar y equilibrar el AVL, haciendo un árbol mucho más compacto y logrando un mejor rendimiento para los comandos (`last` y `date`).

Pareciendo haber resuelto el problema, nos falta una consideración a tener en cuenta, y es a la hora de realizar esta inserción de los Cuacs (nos referimos a sus punteros, como ya hemos recalcado en más de una ocasión) en la lista del nodo, la manera en la que lo hacemos, ya que no basta con agrupar los mensajes dentro de un mismo segundo, deben de mantener un orden determinista para que la salida sea apropiada. En el método `Nodo::addCuac` dentro de `Nodo.cpp`, podemos ver que no realizamos una inserción ciega, si no que vamos recorriendo la lista del nodo con un iterador para insertar el nuevo puntero en su posición correcta respetando el criterio de ordenación que establecimos en `Cuac::es_anterior`.

En el caso de que las fechas sean idénticas, desempataremos por el texto o por el nombre de usuario, para garantizar que el orden de los mensajes sea siempre el mismo y predecible.

Por lo que en conclusión, nuestro árbol resuelve la búsqueda general (llegar a una fecha dada) y la lista en el nodo resuelve la colisión (gestionar si hay más de un mensaje en ese instante).

Aparte de lo explicado anteriormente, se nos pedía que los Cuacs no podían estar duplicados, como consecuencia debíamos usar punteros. La tabla actúa como el almacén de los datos y propietario de los objetos `Cuac`. El árbol actúa como un índice de referencias a esta tabla.

Si nos fijamos, como ya hemos comentado previamente, en la definición de la clase `Nodo` en `Nodo.h` la lista se ha declarado como `std::list<Cuac*> cuacs`. El uso del asterisco es la clave, ya que almacenamos direcciones de memoria (es decir punteros), en lugar de copias de los objetos.

Al existir una única copia física del mensaje (que se encuentra en la tabla), cualquier modificación que pudiera sufrir, se vería reflejada instantáneamente en el árbol sin necesidad de modificarlo también ya que éste solo guarda punteros, por lo que solo habría que modificar la tabla, y actualizar algún puntero en el peor de los casos. Con esto, el consumo de memoria del árbol se mantiene bajo y constante, independientemente del tamaño de los textos almacenados (porque están en la tabla).

Dado que el árbol se construye mediante memoria dinámica, `new Nodo`, es obligatorio liberar esa memoria explícitamente, es decir, una función del código que haga tal, para evitar que a la hora de eliminar un árbol se nos produzca una fuga de memoria, que se nos quede algún nodo suelto sin eliminar.

Para ello hemos implementado una destrucción recursiva. Para el árbol tenemos `~Arbol` cuya única función es invocar a `delete raíz`. Si el árbol está vacío (`nullptr`), la operación es segura y no hace nada.

Para el nodo tenemos `~Nodo`, en el que al ejecutarse el `delete` sobre un nodo, se llama recursivamente a `delete izq` y `delete der` (sus hijos) antes de finalizar, para asegurarnos de que no quede ningún nodo suelto. Con esta estructura recursiva conseguimos que con una sola llamada desde el programa o el destructor de `DiccionarioCuacs`, se desencadene una limpieza ordenada desde la raíz hasta las hojas, liberando la memoria de la estructura del árbol.

Cabe destacar que el destructor del Nodo libera la memoria del propio nodo y de sus hijos, pero no aplica delete sobre los punteros `Cuac*` de su lista interna. Como hemos mencionado anteriormente, los objetos Cuac pertenecen a la `TablaHash` y es responsabilidad de su destructor liberar esa memoria.

Para las rotaciones, se implementaron en la clase `Arbol` (`rotarIzquierda` y `rotarDerecha`) y no en la clase `Nodo`. Si intentáramos implementarlo en la clase `Nodo`, veríamos que un nodo trataría de sustituirse a sí mismo por su hijo para completar la rotación, sin embargo un objeto no puede sobrescribirse a sí mismo porque el puntero `this` es de solo lectura (es inmutable), por lo que movemos esta lógica de rotación al árbol. La función recibe el nodo que causa el desbalanceo, reestructura los punteros y devuelve la nueva raíz del subárbol, que luego se asigna al padre correspondiente. Por ejemplo, vemos cómo en `Nodo * Arbol::rotarIzquierda (Nodo* nodo)`, va a devolver un nodo que es el que se tiene que unir al padre correspondiente y recibe un nodo conflictivo como parámetro. Para realizar la rotación (supongamos `rotarIzquierda`), cogemos el hijo derecho del padre que nos da conflicto (recibido como parámetro), recibe el nombre de `nuevaRaiz`. El antiguo padre desciende a la izquierda, ya que es menor que el hijo derecho, y el hijo derecho (`nuevaRaiz`), sube al padre. Ahora el hijo izquierdo de `nuevaRaiz` queda sin padre, por lo que se pone como hijo derecho del nodo que acaba de descender, manteniendo así la propiedad de orden del árbol binario de búsqueda. Y ya luego con un max entre los dos hijos vemos cual es la nueva altura del árbol tras la rotación. Análogamente ocurre con el `rotarDerecha`. Al principio nos costaba mucho comprender esto, ya que nosotros, al igual que con las tablas estábamos acostumbrados a trabajar sobre papel, y a la hora de implementar una rotación en un programa, se nos complicaba mucho la cosa, por lo que la IA nos sugirió como hacer `rotarIzquierda`, y comprendiendo como se implementaba ésta (explicada previamente para reflejar la comprensión de la función), desarrollamos nosotros la de `rotarDerecha`. Tras devolver la `nuevaRaiz`, permite que el método `insertar` continúe su proceso de inserción, ya que lo de las rotaciones viene a raíz de la inserción de punteros en el árbol con la función `insertar`, y tras comprobar si su factor de equilibrio se ha descompensado o no.

Por último, para los comandos `last` y `date` se sigue la siguiente estructura:

Para `last`, tenemos dos funciones, una que realiza la recursividad, y otra que solo llama a la recursividad y luego imprime en el formato pedido, para simplificar y separar funciones. Hacemos lo mismo con `date`. Esto nos ayudó a organizarnos mejor las tareas que debía realizar cada una y para que fuera más visual.

En `last` (`last_recursivo`), lo que hacemos es que, dado que el árbol está ordenado por fechas de menor a mayor, las más recientes estarán siempre a la derecha, por lo que miramos siempre a la derecha primero con la ayuda de un contador por referencia (`int &contador`), cuando volvemos de mirar a la derecha (`last_recursivo(nodo->getDer(), N, contador);`), miramos el nodo actual por si no hubiéramos llegado al número (`N`) de Cuacs que tenemos que imprimir y hacemos lo mismo, para cada Cuac dentro de la lista del nodo, lo imprimimos en el formato pedido y aumentamos el contador. Por último, si nos quedan todavía Cuacs por imprimir (`contador < N`) miramos en la izquierda.

Para terminar, en el comando `date` (`date_recursivo`), como nos piden imprimir Cuacs en un rango de fechas, lo que hacemos es, como ya hemos mencionado, miramos la derecha los más recientes, si la fecha del nodo actual fuera mayor que la fecha máxima pasada como parámetro, descartamos todo el subárbol derecho, con el izquierdo hacemos lo mismo si la fecha mínima pasada como parámetro fuera menor que la fecha del nodo actual. Una vez comprobamos que está dentro del rango, vemos todos los Cuacs que tienen esos nodos y los imprimimos siguiendo el formato pedido.

5 Variables globales utilizadas (en su caso)

Para completar este apartado, primero es necesario conocer qué es una variable global. Es una variable que es declarada fuera del cuerpo de cualquier función o clase, teniendo un tiempo de vida estático, por lo que es accesible desde cualquier punto del código posterior a su declaración, así como también, su memoria asignada se reserva al inicio de la ejecución del programa y se libera cuando finaliza, manteniendo su valor. Sin embargo, usar variables globales desmedidamente podría provocar efectos no deseados, rompiendo el encapsulamiento que buscamos.

En nuestro proyecto encontramos únicamente dos ejemplos de variables globales, necesarias para la estructura del intérprete de comandos.

En primer lugar, tenemos `const string a[30]`, la cual es un array constante de cadenas de texto que contienen las frases del ejercicio 001 y con lo cual, el texto de los Cuacs. Actúa como tabla de consulta para la traducción entre números y texto. Lo hemos declarado como global debido a que se accede continuamente para las consultas de los Cuacs, por lo que no tendría sentido definirlo como una función ni pasarla como parámetro ya que en cada llamada o en cada declaración obligaría a reconstruir y copiar el array en la pila.

En segundo lugar, tenemos `DiccionarioCuacs dic`, una instancia de la clase controladora principal del programa como es `DiccionarioCuacs`. Debido a que el `main.cpp` del proyecto (en nuestro caso, como ya hemos dicho “`nºdeEjercicio.cpp`”) actúa como un intérprete de comandos, con múltiples funciones como `procesar_mcuac`, `procesar_pcuac`... la instancia del diccionario debe ser compartida por todas ellas. Si no hiciéramos esto, la alternativa sería pasar por referencia esta instancia del diccionario en cada una de las funciones, lo cual complica mucho más la sintaxis del programa.

Cabe destacar que al principio de la construcción del proyecto final (ejercicio 005), se utilizaban variables globales adicionales como `Cuac cuacActual` o `int contador`, pero fueron eliminadas y encapsuladas como atributos dentro de la clase `DiccionarioCuacs` a partir del ejercicio 006.

6 Otras decisiones de diseño relevantes

A continuación, se documenta la evolución del proyecto a través de los distintos problemas resueltos. Se detallarán las decisiones de diseño tomadas y cambios realizados para mejorar la eficiencia y modularidad a medida que iba avanzando el proyecto.

En el ejercicio 001, diseñamos la tabla de mensajes cuac, la cual contenía todos los mensajes de texto que pueden haber. Al principio la definimos como una variable (un array) dentro de una función que precisamente devolvía esta cadena de caracteres, en función del número que recibiera como parámetro, el cual indicaba la posición del array. Se declaró como `const` debido a que era un conjunto de datos inmutable, y que sabíamos que siempre iba a ser así. Decidimos meterla dentro de la función esta declaración ya que como solamente había una función, nos resultaba más sencillo hacerlo de esta manera. Nº de envío Usuario : 51 B68-Antonio Lopez Garcia.

Para el ejercicio 002, este mismo array el cual contenía los 30 mensajes cuac, lo implementamos como una variable global, es decir, fuera de la función, ya que ahora había otra función (`textoAnum`) la cual también hacía uso de esta misma tabla. Lo que hace esta segunda función es a partir de una cadena que recibe como parámetro, va recorriendo este array con 30 entradas con los mensajes cuac, y los va comparando con la entrada, y si en alguna llega a coincidir, devuelve el índice de esa entrada en la tabla. Dado que el tamaño de la tabla es fijo (30), nos podemos permitir una búsqueda lineal $O(N)$, sería complicarnos la vida intentar optimizar esta búsqueda. Nº de envío Usuario : 634 B68-Antonio Lopez Garcia.

Luego pasamos al ejercicio 003, en el que ya se nos pide definir una clase de tipo Fecha, con el

que comenzamos con la programación orientada a objetos. Dentro de este mismo código, lo que hicimos fue definir la clase Fecha que agrupaba los 6 componentes temporales (día, mes, año, hora, minuto, segundo) como atributos privados. Esto nos ayuda con el encapsulamiento y nos garantiza que nadie externo a la clase pueda modificarlos.

Se nos dieron diferentes acciones que debía de realizar esta clase fecha, como leer, escribir... , las cuales se definieron como públicas para que puedan ser usadas en cualquier parte del programa. Primero debíamos construir la fecha, inicializando todos sus atributos a 0. Con leer, comprobamos que la entrada sea correcta, en la que además de leer números, se leen caracteres especiales de la fecha como ‘/’ o ‘:’. Y lo que hacemos para comprobar si el formato es válido es usar separadores, en los que si en algún caso no coincidían o algún valor numérico estaba fuera de rango, devolvía inmediatamente false. La función escribir es de muy fácil comprensión, escribe la fecha usando el formato establecido, y en el caso de que sea un valor menor que 10 imprime un 0 delante. La más importante de este ejercicio es la función de es_menor, la cual recibe por referencia otra fecha, con su año, mes, día... y se van comparando estos campos para determinar si uno es menor que el otro, con una secuencia de condicionales. Por último, la función es_igual que iba comparando campo a campo como la anterior función, viendo si todos coincidían. Nº de envío Usuario : 1931 B68-Antonio Lopez Garcia.

En el 004, el siguiente paso era modelar la unidad básica de información del sistema: el mensaje, conocido como “Cuac”. Esto era necesario ya que el sistema necesita manejar los dos tipos de mensajes que hay (pcuac y mcuac), los cuales tienen asociados un usuario y una fecha. Definimos su clase (Cuac) nuevamente en el mismo código (hasta el momento todavía no nos habíamos planteado dividir el programa en módulos). Esta clase tiene atributos privados, uno haciendo referencia a la clase fecha, y dos string uno con el usuario y otro con el texto del Cuac. Esto nos permite agrupar toda la información del mensaje en un único objeto para que, estructuras posteriores como listas, tablas, árboles, gestionen estos Cuacs completos sin conocer su estructura interna. Para diferenciar los dos tipos de mensaje que tenemos, usamos leer_pcuac o leer_mcuac. En leer_mcuac con un getline leemos el usuario, la fecha y el texto, sin embargo, en el pcuac, tenemos que transformar el número que recibimos en el texto correspondiente con la función numAtexto.

Luego la función de escribirCuac solamente escribe el Cuac en el formato pedido, ayudándose de la función auxiliar fecha.escribir. Y por último, comparamos cronológicamente 2 Cuacs con la función es_anterior en la cual recibimos como en la clase fecha ocurría, otro Cuac, y tenemos que acceder a la fecha de ambos, y con nuestra función definida en fecha de es_menor vamos comparando, y en el caso de que sean iguales estas fechas, en el caso de que el Cuac sea distinto, será un Cuac anterior a otro si su texto es menor y si no comparamos su usuario. Esto mantiene la cohesión alta ya que Cuac sabe quién tiene la fecha, pero Fecha sabe cómo compararla. Nº de envío Usuario : 1978 B68-Antonio Lopez Garcia.

En el ejercicio 005, fue cuando introdujimos la modularidad, separando las clases del programa principal y a su vez, separando sus declaraciones en .h y sus implementaciones en .cpp. En este ejercicio construimos la infraestructura de control que permite al sistema reaccionar a las diferentes peticiones del usuario (pcuac, mcuac, last, follow...). Como no sabíamos el número de comandos que debía interpretar el programa (hasta que dejara de recibir o hasta que recibiera exit), meter toda esta lógica dentro del main resultaría un código mucho más difícil por lo que modularizamos el proyecto de la manera explicada previamente.

En el código del programa principal dejamos únicamente las funciones procesar_mcuac, procesar_pcuac, procesar_last, procesar_follow, procesar_date, procesar_tar y el intérprete. Lo que hace cada una de estas funciones como bien dice su nombre, es procesar la información que necesitan para producir una salida, por ejemplo, en procesar_pcuac, lo que hacemos es con el Cuac que recibimos de la entrada, lo leemos, y ofrecemos la salida siguiendo el formato pedido.

Pero de todo esto lo interesante es el intérprete, ya que según la palabra que reciba (el comando), delega la ejecución a la función especializada en ello. Es por esto que vemos el main mucho más sencillo, el cual solo se limita a leer el flujo de entrada.

Para poder llevar a cabo las funciones su trabajo, era necesario definir una variable de tipo Cuac, que simulaba el Cuac recibido, y un contador para la línea de la salida, que iba aumentando para llevar la cuenta de los mensajes procesados, estas dos variables globales nos servían para comprobar el funcionamiento de la lectura de pcuac y mcuac. Como se puede ver para last o follow se implementaron versiones ficticias que solamente imprimían el cuacActual para verificar que el flujo funcionaba. N^o de envío Usuario : 5820 B68-Antonio Lopez Garcia.

En el ejercicio 006, implementamos un contenedor, es decir una memoria persistente, ya que, en el ejercicio anterior, perdía la información al leer el siguiente comando (se actualizaba tanto el cuacActual como el contador), con esto podemos diseñar una estructura para almacenar un historial de mensajes y poder hacer operaciones sobre éste.

Comenzamos eliminando las variables globales definidas en el programa principal (cuacActual y contador), para dejar únicamente la que hace referencia a DiccionarioCuac, haciendo que el programa principal interactúe únicamente con el diccionario a través de una interfaz como insertar, last, follow, desconociendo como se guardan y se hacen estos procedimientos por dentro. Para las funciones de los cuac (procesar_mcuac o procesar_pcuac) seguía siendo necesario declarar una variable que representara el cuac recibido, pero ahora se hacía cada una en su función, no de manera global.

También tuvimos que definir la clase DiccionarioCuacs la cual se hizo en un .h como el resto. Para esta primera versión de almacenamiento, se eligió una lista doblemente enlazada `list<Cuac> lista`, que era la que se nos sugería. Esta lista permite inserciones en cualquier posición sin necesidad de desplazar elementos en memoria como ocurriría con un vector. También como atributos definimos un contador que se encontrará disponible en toda la clase, utilizado para almacenar el número de elementos insertados en la lista con la función insertar. En esta clase definimos también como públicas todas las funciones que realizará nuestro diccionario como insertar, last, follow...

Primeramente, construimos el diccionario inicializando su contador a cero (obvio ya que todavía no hemos insertado ningún elemento). Luego implementamos la función insertar, la cual optamos por que fuera una inserción ordenada para mantener la lista como tal. Recorre la lista secuencialmente ($O(N)$) con un iterador que en un principio apunta al principio de la lista, buscando la posición correcta con fechas, usando `es_anterior`, explicada en ejercicios anteriores. Lo que hace el bucle es mientras que el iterador no llegue al final de la lista y mientras el nuevo Cuac no sea anterior al iterador, el iterador va avanzando en la lista y actualizando su posición. Buscamos que sea anterior porque como se puede ver, cuando se sale del bucle while, utilizamos el `.insert`, insertando el elemento que se pasa como segundo parámetro (nuestro nuevo Cuac), justo delante del elemento que se pasa como primer parámetro (en este caso nuestro iterador, que vamos actualizando).

Con la función last, hacemos lo mismo, como en el método insertar, íbamos insertando los nuevos mensajes delante de los antiguos, ahora para la función last, como nos piden los más recientes primero, vamos recorriendo esta lista nuevamente con un iterador que comienza en el principio y que avanza o hasta que acabe la lista o hasta que haya imprimido el número de mensajes que ha recibido como parámetro, y en cada iteración vamos aumentando el número de mensajes impresos (llamado como "total") y el índice.

Con el comando follow es donde verdaderamente nos damos cuenta de las limitaciones que tiene este modelo, ya que como bien nos indica follow, se deben listar los mensajes de un usuario dado como parámetro, por lo que para buscar los mensajes de un usuario, debe de recorrer toda la

lista si o si $O(N)$, comparando cadenas de texto para ver si coincide el usuario o no. Para este caso de prueba nos vale pero ya veremos lo que ocurre cuando hay millones de mensajes.

De igual manera nos ocurre con `date`, buscar mensajes en un rango de fechas, teníamos que recorrer la lista entera nuevamente con un iterador que comienza apuntando al principio de la lista, en la cual para cada posición obteníamos la fecha de cada uno `Fecha f = it->getFecha();` y ya veíamos si estaba dentro del rango pasado como parámetro o no con la función que ya tenemos que compara fechas (`es_menor`).

Cabe destacar que para poder realizar este ejercicio, como hemos visto, utilizamos métodos de consulta (getters), por lo que la clase `Cuac.h` ha sido modificada, en concreto la interfaz pública para permitir el acceso a los datos internos del Cuac con `string getUsuario()` y con `Fecha getFecha()`, utilizamos `const` para garantizar que la consulta de estos datos no permita su modificación accidental desde el diccionario. Con esto podemos acceder al usuario y a la fecha del Cuac, imprescindibles para `last` y `follow`. N° de envío Usuario : 5825 B68-Antonio Lopez Garcia.

Para el ejercicio 200, se esperaba que nuestro programa recibiera millones de Cuacs por lo que la implementación anterior no nos valdría, sería muy poco eficiente, es por esto que se nos propone crear una tabla de dispersión para ser capaces de manejar un flujo de mensajes mucho mayor al tratado hasta ahora, con lo que cada vez nos acercábamos más a una aplicación real.

Nuestro objetivo era sustituir la estructura lineal de la que hablábamos en el ejercicio 006, por una de acceso directo para superar las limitaciones de tiempo en la búsqueda de usuarios, ya que demostramos que, por ejemplo el comando `follow`, tenía una complejidad de $O(N)$.

Comenzamos modificando la estructura interna de `DiccionarioCuacs`. Se modificó la clase para reemplazar el atributo `list<Cuac> lista` por una instancia de la nueva clase `TablaHash tabla`, es decir, que el diccionario pasó de ser una lista, a tener una referencia a una nueva Tabla, definida en una clase aparte, no como la lista que estaba definida íntegramente dentro de la clase `DiccionarioCuacs`. Sin embargo, los métodos `insertar`, `follow`, `last`, `date`, se mantuvieron en la parte pública de la clase, con el único cambio de que ahora utilizaban la tabla definida en la parte privada de la clase. Por lo que la clase recibió una remodelación casi al completo.

Además también, si nos fijamos en `DiccionarioCuacs.cpp` que es donde implementábamos nuestro diccionario describiendo su funcionalidad, podemos ver que está vacío, solamente crea el diccionario y nada más, pero porque si nos damos cuenta, `DiccionarioCuacs.h` es el que hace todo junto con la `TablaHash`, entonces la implementación del diccionario la lleva a cabo la propia tabla.

Para este ejercicio específico era validar el funcionamiento y la eficiencia de la tabla creada. Por esta razón se puede ver en el programa principal (`200.cpp`), que los comandos `last` y `date` están comentados, es decir, como si no estuvieran en el programa, ya que como la tabla no tiene orden cronológico, solamente guarda los mensajes, implementar estos dos comandos, hubiera requerido recorrer toda la tabla para acceder a la fecha de cada uno de los Cuacs, $O(N)$, lo cual era ineficiente y era lo que justo buscábamos evitar. Por eso nos centramos en `follow` y en `insertar` (`pcuac` y `mcuac`).

Para la tabla, ya explicada en la sección 4, se trataba de una tabla de dispersión abierta, con atributos privados definidos como su tamaño, el array de cubetas y el numero de elementos, es decir el número de Cuacs que hay en la tabla. También tenemos su función de dispersión, para insertar los elementos y calcular su cubeta, con lo cual, recibirá como parámetro el nombre de usuario. En la parte publica tenemos su constructor y destructor, como se nos pidió para el uso de memoria dinámica, que tuvieran destructor, junto con las funciones de `insertar`, `consultar` y el numero de elementos que hay en la tabla.

Este programa, al no haber sido implementado el árbol todavía, tiene una serie de diferencias

con la tabla explicada en la sección 4, que era la de la tabla final del proyecto.

Primeramente, lo que hacemos es construir la tabla inicializada con 0 elementos y con un array de elementos (la lista).

También declaramos su destructor, que va eliminando cubeta por cubeta.

Para la función insertar, lo que hacemos es primeramente calcular en qué cubeta, es decir en qué posición, va ese usuario dentro de la tabla. Una vez conocida la cubeta con la función de dispersión, implementada justo antes en el código, usando una suma posicional (explicada en la sección 4), accedemos a la lista de dentro de la cubeta, al tratarse de dispersión abierta, `list<Cuac> &cubeta = T[pos]`. Donde pos es la posición calculada en la función de dispersión, y se pasa cubeta por referencia (&), ya que queremos que cualquier cambio que hagamos sobre cubeta, afecte también a T[pos], para que se actualice. Y una vez creada la lista de esa cubeta, accedemos a ella, la recorremos, con un iterador que apunta al principio y avanza con un while mientras no se llegue al final y mientras que el elemento a insertar no sea anterior al elemento al que apunta el iterador (como lo explicado en la sección de la tabla) y entonces ahí es cuando se inserta el Cuac, delante del elemento al que apunta el iterador.

Con la operación consultar, se hace muy parecido, calculamos la posición en la tabla (cubeta) y accedemos a la lista que contiene esta cubeta, para recorrerla e imprimir solamente los Cuacs del usuario pasado como parámetro, con el formato pedido, por eso el uso de las variables indice y total.

Cabe destacar que usamos unsigned por lo explicado antes, y que la función consultar equivale a la de follow. N^o de envío Usuario : 5826 B68-Antonio Lopez Garcia.

Ejercicio 300, en el que la cosa ya comienza a parecerse más a lo que buscábamos desde un principio. Se trata de un punto de inflexión en el proyecto, ya que pasamos de tener solamente almacenamiento (en la tabla), a tener indexación (tabla y árbol).

Una vez resuelta la búsqueda por usuario en el ejercicio anterior, como bien hemos comentado, al no presentar orden cronológico, manteníamos ese problema de tener que recorrer la tabla completamente para comandos como last o date, en los cuales se requería la fecha de cada Cuac, lo cual hacía del programa, un proyecto ineficiente hasta el momento.

Es aquí donde introducimos los árboles, ya que la tabla dispersaba los datos aleatoriamente, perdiendo el orden temporal, por lo que era necesario introducir una estructura auxiliar que mantuviera los mensajes ordenados sin duplicar la información. Esto último es de lo más importante que tiene este árbol.

En primer lugar, modificamos la clase DicionarioCuacs.h, al igual que hicimos con las tablas, implementamos el uso del árbol dentro del diccionario, justo debajo de la tabla, y para funciones que lo requiriesen, por ejemplo last, date, o insertar, utilizamos el árbol. Insertar es un caso particular ya que se necesita de esta función en ambas estructuras, entonces lo que se nos propuso, es lo que se puede ver, que insertamos un elemento en la tabla y guardamos su referencia, es decir, su dirección de memoria definitiva (la del objeto), y este puntero es el que insertamos en el árbol. Es por esto por lo que decía antes que es de las partes más importantes del árbol, que no duplica elementos, si no que guarda referencias a la tabla para una mayor simplicidad y que ambas estructuras estén sincronizadas.

Tuvimos que crear también una clase Nodo.h para cada nodo del árbol, el cual para cumplir con la no duplicidad de datos, se creó un nodo que almacenara una lista de punteros `std::list<Cuac*> cuacs`, en lugar de objetos Cuac directamente. Esto también nos ayuda a usar menos memoria, ya que almacenar solamente referencias, ahorramos muchísima memoria. También definimos lo propio del nodo (en privado como siempre), lo esencial que era la fecha, para lo que hacíamos el

árbol, también sus hijos y la altura de ese nodo para mantener el balanceo.

Luego también tenemos su constructor y destructor, ya que usa memoria dinámica y ya los métodos de acceso necesarios como los getters y setters, para poder realizar las funciones propias del árbol. Además, las listas de los nodos sirven para guardar varios elementos ocurridos en el mismo instante, y ahorrarnos mucha altura del árbol y simplicidad a la hora de operar.

Para la implementación del nodo, como siempre lo construimos con sus hijos y la fecha que contiene, de altura 1. También definimos su destructor, que hace que antes de eliminar ese nodo, elimina sus hijos para asegurarnos. Y ya vamos implementando todos los métodos de acceso. La única función algo compleja de nodo es la de insertar un Cuac dentro del nodo, en su lista, pero lo hacemos como hacíamos con la tabla, con un iterador que vaya recorriendo la tabla desde el principio, y con `es_anterior` buscamos su posición correcta. Usamos `c->es_anterior(**it)`, la flecha porque `c` no es el objeto en sí, es la dirección de memoria del objeto, y luego usamos `(**)`, porque `it` apunta a un puntero y nosotros queremos acceder al objeto.

Luego definimos la clase `Arbol.h`, que como dijimos se trata de un árbol AVL, en el que en sus atributos ponemos su raíz, un puntero que apunta a un nodo, y ya su altura, su factor de equilibrio para ver si está desbalanceado, insertar y la función de `last` (`last_recursivo`). Definimos en privado insertar y `last` para que no se pueda modificar el funcionamiento de estas funciones, y en público solamente ponemos como la fachada, las funciones que llaman a estas funciones internas, además de ser más seguro, nos era más fácil de entender visualmente. En público tenemos las rotaciones y las fachadas de las que hemos hablado.

La implementación del árbol ya está explicada, aquí solamente vamos a hacer un rápido resumen. Comenzamos construyendo el árbol, y con la recursividad, lo destruimos, ayudándonos del destructor del nodo. Luego vemos la altura del árbol con el método de acceso, necesaria para calcular el factor de equilibrio y ver si está desbalanceado o no, lo cual comprobaremos después de cada inserción, por eso en factor equilibrio no se manda a hacer ninguna rotación, porque inicialmente el árbol está vacío, solo se modificará con las inserciones, que es donde miramos si hay desbalanceo, y mandamos a hacer rotaciones, definidas también en el código. Y ya vamos actualizando la altura de cada nodo y del árbol también. Por último, implementamos la función `last`, motor junto con `date` (del ejercicio 301) para implementar los árboles.

Esta función `last` como ya hemos explicado, va en orden inverso empezando por la derecha y escribiendo los mensajes más recientes, llevando la cuenta de los impresos con un contador y mientras que sea $< N$ va recorriendo en orden, derecha, nodo actual, izquierda, y se reduce su $O(N)$ siendo ese N solamente el pasado como parámetro en un árbol con millones de nodos.

En `300.cpp` comentamos el resto de las funciones ya que nuestro objetivo era comprobar únicamente el funcionamiento del árbol con el comando `last` como pedía el ejercicio, y que estuviera bien implementado junto con la tabla.

Si nos fijamos bien, en la clase de la tabla (`TablaHash`), la función de insertar ha pasado de `void TablaHash::insertar` a `Cuac* TablaHash::insertar`, esto es porque como hemos visto, el árbol necesita insertar la referencia que devuelve esta función. Como para pasar el Cuac de la tabla al árbol, el diccionario no puede duplicar elementos (perderíamos la eficiencia que hemos conseguido con el árbol), por lo que tiene que darle la dirección de memoria de donde lo guardó la tabla.

Cuando nosotros insertamos con `cubeta.insert`, esto devuelve un iterador (`it_res`) que señala a este elemento recién guardado. Como el iterador es un objeto que actúa como un puntero, no nos vale para el árbol porque necesita un puntero puro `Cuac*`. `It_res` es el iterador que señala al objeto recién añadido (el que queremos pasar al árbol), con el `*` accedemos al objeto real (al Cuac que señalaba), y con la referencia `&` obtenemos la dirección de memoria del objeto, que

era lo que buscábamos para pasarle al árbol. Esto permite que el diccionario reciba la ubicación física definitiva del objeto en la tabla y se la pase al árbol, garantizando que el árbol apunte a ese dato almacenado en la tabla. N° de envío Usuario : 6069 B68-Antonio Lopez Garcia.

Una vez realizado el ejercicio 300, habiendo implementado el árbol, realizar el ejercicio 301 resulta bastante sencillo ya que únicamente es implementar el comando `date`, para un rango de fechas, mostrar los Cuacs que se encuentran en ese rango. Como ocurría con el comando `follow`, con la tabla requeriría acceder a toda la tabla para poder representar estos mensajes en el intervalo pedido, $O(N)$.

Como se puede ver en este ejercicio es todo reutilizado del ejercicio anterior, lo único que cambia es dentro de la clase árbol, implementar nuestro `date` y `date_recurso`, que como ya dijimos, la verdadera funcionalidad se encuentra en `date_recurso` para protegerlo y que `date` solamente se encargaba de llamar a la función e imprimir conforme al formato especificado.

Esta función ya fue explicada en la sección 5 por lo que aquí simplemente haremos un repaso general.

Nuestro `date_recurso` mira si nuestra fecha actual ya es mayor o igual que la fecha del límite superior del rango con `es_menor` para ver si nos podemos descartar el subárbol derecho. Además, miramos primero por la derecha que son los más recientes. Si se descarta el subárbol derecho miramos el actual con las condiciones antiguo y nuevo para comprobar que ese nodo esté dentro del rango, y una vez que estamos dentro del nodo imprimimos su lista (recordemos que cada nodo es una lista de punteros a cuacs). Una vez miramos el nodo actual, vamos al izquierdo y evaluamos como antes, si nuestra fecha es menor que el rango mínimo nos podemos descartar el subárbol izquierdo.

En 301.cpp activamos nuevamente el comando `date`, y dejamos el `follow` comentado ya que aquí estamos viendo los que afectan directamente al árbol. N° de envío Usuario : 6070 B68-Antonio Lopez Garcia.

Por último el 302 es la versión final del proyecto, cuyo objetivo era integrar todos los módulos en una aplicación y habilitando todos los comandos. Ya habíamos realizado entregas aisladas para el programa como el 200, 300, 301, ahora iba todo junto, por lo que aquí no hay cambios, si no que todo va activado en conjunto. N° de envío Usuario : 6071 B68-Antonio Lopez Garcia.

7 Resumen de tiempo dedicado a cada apartado

Se trata de un ejercicio complejo y largo, ya que se va haciendo poco a poco a lo largo de las sesiones de prácticas. Primero comenzamos con programas muy sencillos, pero a medida que íbamos avanzando se nos iba complicando más y más.

Para los primeros problemas del 001-004 alrededor de 1 hora o menos cada uno de ellos ya que simplemente era definir las primeras funciones y dos clases, cosa que no requería de mucho tiempo.

Luego a partir del 005 junto con el 006, aunque no fueran excesivamente complejos ni largos, nos costó también el hecho de que tuviéramos que implementar la modularidad con los `.h` y `.cpp`, y a la hora de hacer el `Makefile`, que al principio nos daba error. En estos dos ejercicios estuvimos más tiempo, de unas 4 a 6 horas para hacerlos, además de que teníamos que hacer el intérprete de comandos y la primera versión de `DiccionarioCuacs`.

En el 200, se nos complica bastante la cosa ya que comenzamos a implementar las tablas, y con las valoraciones de que tipo de dispersión es mejor, luego diseñar la fórmula de dispersión, y realizar la implementación de toda la tabla, nos supuso bastante tiempo ya que nos resultaba

bastante difícil realizar todo esto. Se estima que estuvimos cerca de las 8-10 horas para realizar este ejercicio completo.

El 300 para nosotros sin ninguna duda ha sido el más complejo y laborioso de todos, ya que implementar el árbol suponía actualizar tanto en cómo se insertaban elementos en la tabla, como definir cada nodo del árbol, y también el árbol en sí, determinar su tipo, implementarlo... Estimamos que estuvimos en total unas 15 o 16 horas para realizarlo, debido a su extensión y complejidad.

Por último el 301, el cual solamente era aplicar el comando `date`, estuvimos bastante poco, cerca de las 2 horas, ya que estaba prácticamente hecho ya, solamente tuvimos que diseñar el `date` y `date_recursivo` e implementarlo en el programa principal.

El 302, no sabemos cuanto tiempo fue porque realmente es juntar todo lo que tenemos hecho entonces no lo consideramos como ejercicio, si no como comprobar que todo lo que tenemos hecho funciona de manera conjunta.

La redacción de esta memoria, como se puede ver es algo extensa y detallada, por lo que nos ha llevado algo más de tiempo de lo esperado, cerca de las 6 u 8 horas, pero queríamos asegurarnos de no dejar nada sin detallar ni explicar. A continuación, explicamos unas breves conclusiones a las que hemos llegado con la realización de este proyecto.

8 Conclusiones

La realización del proyecto Cuacker ha supuesto un punto de inflexión en nuestra comprensión en las estructuras de datos, ya que, aparte de las dificultades superadas a la hora de desarrollar código en C++, hemos podido comprobar la importancia que tiene la elección de la estructura de datos adecuada en el rendimiento de una aplicación.

Hemos visto como pasando de estructuras lineales (006) a estructuras avanzadas como tablas y árboles, no es por capricho, si no una necesidad para poder escalar el funcionamiento del proyecto, ya que como hemos comprobado, al recibir millones de entradas, el programa se colapsaba sin estas estructuras. También hemos utilizado la memoria dinámica y el uso de punteros. A lo que podemos llegar es que con un buen diseño, con módulos, encapsulamiento, fachada (diccionario-Cuacs) es tan importante como la estructura de datos utilizada y sus funciones para construir un programa eficiente.